

Cross-Signal Anomaly Detection for Enterprise AI-Agent Governance:

A Taxonomy of Agent Misbehavior Classes and a Synthetic-Telemetry Evaluation Framework

Draft — for review

Abstract

Enterprise AI agents execute tasks autonomously and emit rich execution telemetry: tool calls, latencies, token and cost accounting, permission requests, escalations, retries, and outputs. Today’s governance regimes are mostly post-hoc: audits, logs, and after-the-fact review. This paper asks whether such telemetry supports pre-violation detection — flagging agent misbehavior before the violating action completes — and builds the machinery to answer that question reproducibly. We contribute (i) a machine-readable telemetry event schema for agent governance; (ii) a taxonomy of nine agent misbehavior classes (scope creep, privilege-escalation attempts, runaway loops, data exfiltration, prompt-injection compromise, cost anomalies, silent failure masking, approval-gate circumvention, and cross-agent collusion), each defined behaviorally with observable signals, a violation-completion point, and a parameterized injection for instantiation; (iii) a seeded synthetic telemetry generator that emits $\sim 50k$ sessions with logged ground truth; and (iv) a detection benchmark comparing four detector families — static rule thresholds (D1), per-agent statistical baselining (D2), Isolation Forest on session feature vectors (D3), and a Markov transition-likelihood sequence model (D4) — on per-class ROC/PR, false-positive cost per agent-day, and detection lead time. On the synthetic benchmark at a headline budget of 0.1 false positives per agent-day, D2 gives the best overall ranking quality (ROC-AUC 0.93) and detects 70% of injected anomalies with a median of 21 s of lead time before violation completion; D1’s interpretable rules capture 63%; D3 captures 41%; the sequence model D4 ranks weakly overall but carries unique signal on behavioral discontinuity. Coverage varies sharply by class: the benchmark honestly exhibits the designed gaps — aggregate detectors are essentially blind to mid-session prompt-injection compromise (recall ≤ 0.02), and cross-agent collusion eludes session-scoped detectors by construction. No production validation is claimed; all results come from synthetic telemetry with illustrative parameters.

1 Introduction

Autonomous agents are moving from demos into enterprise workflows: they read and write documents, query databases, call external APIs, and escalate to humans when policy demands. With autonomy comes an execution surface that traditional governance was not designed for. A conventional audit asks after the fact what an actor did and whether it was permitted; an agent, however, can complete hundreds of micro-actions per hour, and the actions that matter — an out-of-scope write, an egress of sensitive records, a loop that burns budget — may complete in seconds. Post-hoc review of agent activity is therefore doubly weak: it arrives after the harm, and its volume defeats manual inspection.

A different posture is available. Agent runtimes already stream telemetry: every tool invocation, every permission request and its outcome, every retry and escalation, every token and dollar spent. If that stream is scored online, governance can shift from audit to early warning: flag a session whose behavior departs from the agent’s established norm while there is still time to intervene, gate, or revoke. This paper studies that shift, with three questions:

- Q1 (representation). What telemetry schema faithfully captures agent behavior at a level that supports anomaly detection? (Section 3.)
- Q2 (taxonomy). What classes of agent misbehavior are observable in that schema, and how does each class manifest? (Section 4.)
- Q3 (detection). How do four practical detector families rank and cover these classes, what do they cost in false alarms per agent-day, and how much lead time do they buy before a violation completes? (Sections 5–7.)

Position within the literature. LLM-agent safety research predominantly evaluates task success and harm in interactive benchmarks (AgentBench [9], ToolEmu [14], τ -bench [15], InjecAgent [18]); insider-threat and host-intrusion detection research has long studied behavioral anomaly scoring, including Markov chain models of user behavior [16, 17]; AIOps has industrialized log-anomaly detection [5]; and drift detection formalizes how baselines decay [2]. What is missing is a bridge: a governance-oriented telemetry model and misbehavior taxonomy for enterprise agents, evaluated online with a benchmark whose ground truth is fully known. That bridge is this paper’s contribution.

Contributions.

1. A machine-readable telemetry event schema defining the governance-relevant event stream (Section 3; artifact: `artifacts/telemetry_event_schema.json`).
2. A taxonomy of nine misbehavior classes, each with a behavioral definition, observable signals, severity weight, an explicit violation-completion point (the anchor for lead-time scoring), and a parameterized injection (Section 4; artifact: `artifacts/misbehavior_taxonomy.json`).
3. A seeded synthetic generator producing $\sim 50k$ sessions across 500 agents with injected anomalies and logged ground truth — a reusable, byte-reproducible benchmark (Section 5).
4. A detection benchmark comparing four detector families on per-class ROC/PR, alert cost per agent-day, and detection lead time, including a deliberately reported coverage-gap matrix (Sections 6–7).

Scope and honesty statement. The evaluation is entirely synthetic. The generator encodes the anomaly classes it later asks detectors to find, so measured performance is an upper bound on what the same detectors would achieve on real telemetry. We deliberately keep some classes subtle — prompt-injection compromise is invisible to aggregate detectors by construction, and cross-agent collusion is mostly out of reach of session-scoped detection — and report those gaps rather than tuning them away (Section 8). No ServiceNow product internals, customer data, or proprietary telemetry are described or used; all parameters are illustrative, informed only by public literature.

2 Related Work

LLM agent safety and security. Public frameworks name the risks: the NIST AI Risk Management Framework [12] organizes govern/map/measure/manage functions around scoped authority and human oversight, and the OWASP Top 10 for LLM Applications [13] enumerates excessive agency, sensitive-information disclosure, prompt injection, and unbounded consumption — four classes our taxonomy instantiates directly. Indirect prompt injection demonstrated that agents can be steered by content they consume [3], and InjecAgent benchmarks the attack surface for tool-integrated agents [18]. MITRE ATLAS [11] catalogs adversary tactics against AI systems, including exfiltration. Agent benchmarks such as AgentBench [9], ToolEmu [14], and τ -bench [15] measure capability and harm in interactive settings. Our work is complementary: rather than scoring harm in the loop, we score the telemetry that any of these systems already emits, and we contribute the governance-specific taxonomy that connects them.

Agent governance. Recent legal and policy work argues that agents require governance mechanisms beyond model-level alignment: ex-ante constraints on the principal-agent relationship, runtime oversight, and liability allocation [7]. Multi-agent deployments additionally create systemic risks — collusion, coordination, and conflict — that are properties of populations, not individuals [4]. Our cross-agent collusion class is a direct, telemetry-observable instantiation of the latter concern, and our lead-time framing is a quantitative version of the ex-ante/runtime contention.

Insider-threat and host-intrusion detection. The closest methodological ancestors are behavioral intrusion detection: early work modeled user command sequences as Markov chains and flagged low-likelihood transitions [16], generalized to dynamic/static behavioral models [17]. Insider-threat surveys [6] organize detection around deviations in access and activity patterns. These methods operate on command or API streams quantitatively similar to our tool call stream; the differences we introduce are the governance-oriented event schema (permission requests, escalations, approval outcomes), the misbehavior taxonomy, and the lead-time evaluation protocol against known violation-completion points.

AIOps anomaly detection, outlier detection, and drift. Log-anomaly and anomaly-detection pipelines in AIOps are surveyed extensively [5]; Isolation Forest [8] remains a standard unsupervised baseline for multivariate point anomalies; drift adaptation [2] formalizes why static baselines decay. Our detector set is deliberately conventional — a rule baseline, robust per-agent z-scoring, Isolation Forest, and a Markov sequence model — because the contribution is the evaluation framework and taxonomy, not a novel learner; the framework is designed so stronger detectors can be dropped in behind the same interface.

Precision/recall under class imbalance. For rare-class evaluation we report both ROC and precision-recall curves following Davis and Goadrich [1], and we anchor operating points to a budget in false positives per agent-day rather than a bare threshold, because in operations the alert rate is the binding constraint.

3 Telemetry Model

The telemetry model is the citable interface between an agent runtime (or gateway) and any downstream detector. We define one event = one completed action, with the following fields (artifact: `artifacts/telemetry_event_schema.json`)

Field	Type	Semantics
<code>session_id</code>	string	one task instance executed by one agent
<code>agent_id</code>	string	static agent identifier
<code>ts</code>	float	completion time, epoch seconds; strictly increasing within a session
<code>event_type</code>	enum	{ <code>tool_call</code> , <code>permission_request</code> , <code>escalation</code> , <code>retry</code> , <code>output</code> }
<code>tool_name</code>	string?	invoked tool (null for output)
<code>latency_ms</code>	float	wall-clock duration of the action
<code>tokens</code>	int	tokens attributed to the event
<code>cost</code>	float	USD attributed to the event
<code>target_scope</code>	string	family:name resource scope (e.g. db:dev)
<code>approved</code>	bool?	approval outcome for <code>permission_request</code> /escalation

Three modeling choices carry the governance semantics:

1. Permission events are first-class. A denied `permission_request` is evidence of an agent attempting to exceed its grants — a governance signal with no analogue in classical log schemas. For a gated tool call, the detector can reconstruct whether an approval preceded it (a gate-consistent call), followed it (post-hoc approval), or never happened (approval gap).
2. Scopes are declared and comparable. Each agent declares a scope set; an event whose `target_scope` lies outside it is an out-of-scope action by definition, and an ungated tool acting on a scope outside its tool family is a proxy bypass candidate.

Table 1: Violation-completion anchors used for lead-time scoring.

Class	Violation completes when ...
Scope creep	first out-of-scope action completes
Privilege-escalation attempts	4th denied permission request completes
Runaway loops / cost anomalies	cumulative cost first exceeds $3\times$ agent baseline
Data exfiltration	last egress event completes
Prompt-injection compromise	final action of the hijacked pivot window completes
Silent failure masking	terminal output emitted while unescalated failures remain
Approval-gate circumvention	last circumventing action completes
Cross-agent collusion	second agent’s final coordinated action completes (handoff occurred)

3. Retries and escalations are separated. Retry counts measure silent failure absorption; escalations measure failure surfacing. Their ratio is a governance signal (silent failure masking).

The subsequent detector suite is developed against this schema only. Sessions with output terminate normally; a session is the unit of scoring, with events scored online as a growing prefix (Section 6).

4 Misbehavior Taxonomy

The taxonomy is the paper’s core conceptual contribution: nine classes of agent misbehavior that are observable in governance telemetry and stably defined across implementations. For detection lead time we additionally require each class to define an unambiguous instant when the misbehavior completes — without it, “early warning” has no referent. Severity weights are illustrative ordinals (1–5), not calibrated risk figures. Artifact: `artifacts/misbehavior_taxonomy.json`.

Class	Behavioral definition	Primary signals	Sev.
Scope creep	agent acts on resources outside its declared task scope; out-of-scope fraction grows	out-of-scope ratio/count, novel scopes	3
Privilege-escalation attempts	repeated denied permission requests accumulating within a session	denied count/ratio, new tools requested	4
Runaway loops	short tool cycle repeated far beyond necessity until budget boundary	pattern repetition, call count, cost vs baseline	3
Data exfiltration	read-heavy burst on sensitive scopes followed by external writes	egress count, egress token share, out-of-scope ratio	5
Prompt-injection compromise	mid-session behavioral discontinuity: foreign transition regime, aggregates unchanged	split-half log-likelihood drop, min- k transition likelihood	5
Cost anomalies	token/spend drift vs task-class baseline with unchanged call structure	tokens/cost per call vs baseline	2
Silent failure masking	high retry counts, suppressed escalation, success-claiming output	retry ratio, zero escalations	3
Approval-gate circumvention	gated actions executed without prior approval: missing, post-hoc, or via proxy tools	approval gaps, post-hoc approvals, proxy bypasses	4
Cross-agent collusion	complementary halves of a violating pattern split across an agent pair in a short window	pairwise timing, shared novel scope, complementary roles	5

Violation completion anchors. Table 1 summarizes per class. These anchors drive the headline metric: detection lead time = violation_ts – first detection timestamp. Positive lead time means the detector fired while the violation was still avoidable.

Two design commitments distinguish the taxonomy from a list of attack types. First, evidence lives in different signals for different classes: scope discipline for scope creep, denial dynamics for privilege escalation, timing and volume for loops and cost drift, sequence structure for prompt injection. A single detector cannot cover all nine, which is precisely why the benchmark reports a class $\times$ detector matrix (Section 6). Second,

the taxonomy names its own blind spots: prompt-injection compromise is defined by construction to leave aggregates unchanged, so we predict near-total blindness for aggregate detectors and partial visibility for sequence models; the benchmark then confirms it rather than hiding it.

5 Synthetic Telemetry Generator

Real governance telemetry with labeled misbehavior does not exist in public form, and a benchmark for pre-violation detection requires known ground truth for when each violation completed. We therefore build a seeded synthetic generator (`code/generate_agents.py`, `code/generate_telemetry.py`) whose parameters are illustrative but whose structure follows the taxonomy. Synthetic-data methodology is standard [10]; the novelty here is the governance-aware injection layer.

Agent population. 500 agents across six task classes (doc drafting, data analysis, customer support, code maintenance, compliance audit, IT operations). Each agent receives: a task class with a canonical workflow; a per-agent first-order Markov chain over its permitted tools (the class template perturbed by a Dirichlet draw, masked to the agent’s permissions); declared scopes (`workspace:*`, `db:dev`, `sandbox:*`, ...); a session rate (4–18 sessions/day); and an analytic cost baseline derived from its own stationary distribution and the illustrative price table.

Benign sessions. Session lengths follow a clipped negative binomial (mean ≈ 7 tool calls); tool sequences are sampled from the agent’s own chain; latencies and token counts are lognormal per tool; permission dynamics include approved requests, denials with escalation, rare cached approvals, and a small benign retry rate. Benign sessions are therefore noisy on purpose: approximately 8% of benign sessions carry retry-heavy patterns, and 12% contain at least one denied request — the false-positive pressure that makes the operating-point comparison meaningful.

Anomaly injection. Each of the nine classes is a parameterized mutation of a benign session, logged to ground truth with its violation timestamp and the parameters used. The injectors are written to materialize the class they represent: e.g. exfiltration appends a read burst on sensitive scopes plus external writes (token-inflated) and anchors the violation at the last egress; prompt-injection compromise re-samples the pivot window from the most KL-divergent foreign task-class chain restricted to permitted, non-egress tools, draws 12 candidate realizations and commits the one most divergent under the agent’s own chain, preserves all per-call latency/token/ cost draws, family-consistently re-scopes tools, and inserts gate-consistent approvals where needed — so that by construction no aggregate or policy rule trips; runaway loops append a repeating cycle (length 1–3, repeated 10–24 times); silent-failure masking injects a retry storm and deletes escalation events, then verifies the session has at least the minimum retry count of the injected pattern (injecting additional retries on insufficient sessions) so the class always materializes as defined.

Corpus and reproducibility. The principal corpus (seed 42) contains 49,377 sessions and 525,123 events over 10 simulated days (≈ 9.9 sessions/agent-day), with 985 injected anomalies (2.00%; 108–111 per class). Sessions are split 60/40 into calibration (29,622) and evaluation (19,755) sets; all injections land in the evaluation split, the calibration split is verified anomaly-free, and detectors are only ever fit on calibration data. Regeneration is byte-reproducible: SHA-256 checksums of both generated CSVs are recorded in `data/generation_metadata.json` and verified by re-running the pipeline (identical checksums).

6 Detection Pipeline and Evaluation

Feature extraction. All detectors consume one shared feature vector per session (`code/features.py`), computed to be well defined on any prefix of a session just as on a complete one — so a threshold calibrated on complete sessions applies unchanged to online prefixes. The 39 features span volume/cost, latency, permission behavior, retries/escalations, scope discipline, gate discipline (approval gaps, post-hoc approvals, proxy

Table 2: Overall evaluation on the held-out split (985 anomalies / 18,770 benign sessions; all thresholds out-of-sample). FP/ad = realized false positives per agent-day at the operating point.

	ROC-AUC	PR-AUC	Recall	Precision	FP/ad	Lead time (pre-viol. rate / median)
D1 static rules	0.890	0.686	0.625	0.738	0.115	0.48 / 21.9 s
D2 per-agent baseline	0.934	0.786	0.702	0.748	0.123	0.70 / 21.2 s
D3 isolation forest	0.900	0.547	0.410	0.676	0.102	0.31 / 11.2 s
D4 Markov sequence	0.629	0.116	0.078	0.298	0.095	0.10 / 12.7 s

bypasses), repetition structure, and sequence likelihood under the agent’s own calibrated chain (mean step log-probability, min-3 step mean, split-half drop).

The four detector families.

- D1 — static rule thresholds (operational baseline): a weighted sum of normalized exceedances over interpretable rules (out-of-scope ratio, denied-request count, retry-ratio-with-no-escalation, egress token share, budget ratio, pattern repetition, gate bypasses).
- D2 — per-agent statistical baselining: robust z -scores of each governance signal against the agent’s own history (scale = $\max(\text{IQR-based}, \text{per-signal floor})$), session score = max positive z . Agents with insufficient history fall back to task-class statistics; per-signal floors are required because several governance signals are zero-inflated (a robust scale collapses to zero).
- D3 — Isolation Forest [8] on session feature vectors (log-transformed heavy-tailed features, standardized), trained on mixed-length prefix rows so that the model is meaningful when applied to partial sessions.
- D4 — Markov transition-likelihood sequence model: per-agent first-order chains over tool calls (Dirichlet-smoothed), score = max of two robust z -scored statistics — min-3 step log-likelihood (foreign transition windows) and split-half drop (mid-session regime change).

Protocols. Detectors are fit on 75% of the calibration split; operating thresholds are selected on the remaining 25% (out-of-sample) to match a target alert budget in false positives per agent-day; the evaluation split is then scored by those same models, so the reported operating points are in-sample nowhere. Lead time is measured by rescoring each anomalous session on growing prefixes with the same feature function, recording the first prefix timestamp at which the operating threshold is crossed, and comparing it with the ground-truth violation timestamp. Detection rate counts sessions crossing at any prefix (a slightly permissive protocol than end-of-session classification and reported as such); ROC/PR is computed end-of-session, one class versus benign, on the evaluation split.

Headline results. Table 2 reports the overall comparison at the headline budget (0.1 FP/agent-day). D2 leads overall: with per-agent baselining, 70% of injections are detected with a median of ≈ 21 s of lead time before the violation completes, at an alert cost of 0.12 FP/agent-day. D1’s static rules trail modestly (63% recall), confirming that hand-set policies retain real signal once count gates and benign-noise floors are tuned honestly. D3’s unsupervised scoring generalizes worse at this alert budget, and D4 — with no aggregate features at all — ranks near chance overall, exactly as a sequence-only detector should on a corpus dominated by aggregate anomalies.

Per-class coverage. Table 3 is the paper’s coverage-gap matrix: recall per class at the headline budget, with ROC-AUC in parentheses. Figure 1 renders the same matrix as a heatmap.

Four findings stand out:

1. Exfiltration is caught universally and early. All of D1–D3 reach recall 1.00 with AUCs ≥ 0.99 , and every pre-violation detection lands ≈ 19 s before the data leaves the boundary — the read burst is visible long before the egress completes.

Table 3: Class $\times$ detector recall at 0.1 FP/agent-day (ROC-AUC in parentheses). Bold: best per row.

Class	D1	D2	D3	D4
Scope creep	0.48 (0.81)	0.59 (0.95)	0.02 (0.82)	0.04 (0.50)
Privilege escalations	0.55 (0.91)	1.00 (0.98)	0.35 (0.96)	0.02 (0.52)
Runaway loops	0.93 (0.98)	1.00 (0.99)	0.72 (0.98)	0.05 (0.65)
Data exfiltration	1.00 (0.999)	1.00 (0.99)	1.00 (0.998)	0.30 (0.84)
Prompt-injection comp.	0.01 (0.47)	0.01 (0.51)	0.01 (0.49)	0.02 (0.66)
Cost anomalies	0.22 (0.94)	1.00 (0.99)	0.86 (0.98)	0.03 (0.52)
Silent failure masking	0.85 (0.93)	0.43 (0.92)	0.05 (0.83)	0.00 (0.48)
Gate circumvention	0.87 (0.92)	0.56 (0.93)	0.48 (0.97)	0.18 (0.74)
Cross-agent collusion	0.73 (0.90)	0.73 (0.96)	0.23 (0.92)	0.06 (0.71)

2. Session-level detectors structurally miss cross-agent collusion. The class is built so that each participating session is only mildly anomalous; D1/D2 still catch 73% at the headline budget (via the shared out-of-scope destination) but at the cost of very long lead times, and D3 collapses to 0.23. Pair-level scoring is genuinely required; a session-scoped pipeline cannot fully solve this class.
3. Prompt-injection compromise is the designed blind spot. D1–D3 are at AUC ≈ 0.5 (chance) because by construction no aggregate or policy rule is tripped. D4 carries the only signal (AUC 0.66) — the split-half drop and min-3 statistics see the regime change — but even D4 converts only a sliver to recall at strict alert budgets. This gap is real and reported, not tuned away: behavioral-discontinuity detection at operational alert rates remains open.
4. Detector strengths are complementary, not redundant. D1 owns silent-failure masking (0.85) and gate circumvention (0.87) where explicit policy structure dominates; D2 owns scope creep, privilege escalation, loops and cost drift where per-agent baselining dominates; D3’s only clear edge is cost anomalies (0.86); D4 is the only detector with any prompt-injection signal. A production ensemble would combine these signal families rather than pick a winner.

Lead time. Figure 3 reports the lead-time distributions. Among pre-violation detections, D2 buys a median 21.2 s of warning (D1: 21.9 s; D3: 11.2 s; D4: 12.7 s). Lead time varies sharply by class: loops and gate circumvention yield short, tight warnings (the violating action is the anomalous action), while collusion yields the longest median lead (≈ 164 – 306 s) because the pair’s handoff gap is itself the signal. Cost anomalies and privilege-escalation bursts sit in between, reflecting how late their completion anchors fall.

7 Sensitivity Analysis

Three analyses probe the stability of the conclusions (code/sensitivity.py; results in results/sensitivity.json).

Prevalence sweep. The corpus is regenerated at anomaly prevalences $\{0.5\%, 1\%, 2\%, 5\%, 10\%\}$. Benign sessions are identical across prevalence levels (only mutation coverage changes), and thresholds are calibrated on benign data at a fixed alert budget, so this sweep isolates how detection behaves as misbehavior becomes rarer or more common. Ranking quality is remarkably flat: D2 holds ROC-AUC 0.928–0.934 across the whole range, D1 0.880–0.890, D3 0.894–0.900. Recall at the fixed budget is likewise flat (D2: 0.69–0.72; D1: 0.60–0.62; D3: 0.38–0.40; D4: 0.06–0.08) — the expected consequence, and a useful discipline, of budgeting on false positives rather than on a fixed threshold: at fixed FP/agent-day, rarity does not degrade recall, it degrades precision (the analyst-facing cost).

Seed stability. The full pipeline (population $\rightarrow$ telemetry $\rightarrow$ injection $\rightarrow$ detection) is re-run under seeds 101/202/303. Overall AUCs shift by at most 0.02 (e.g. D2: 0.934 $\rightarrow$ 0.924–0.934). The coverage-gap ranking is stable for the aggregate detectors (Spearman ρ of per-class recall vs. the principal run: 0.95–1.00 for D1–D3) but noisier for D4 (0.72–0.78), which is unsurprising for a detector operating in a low-recall regime

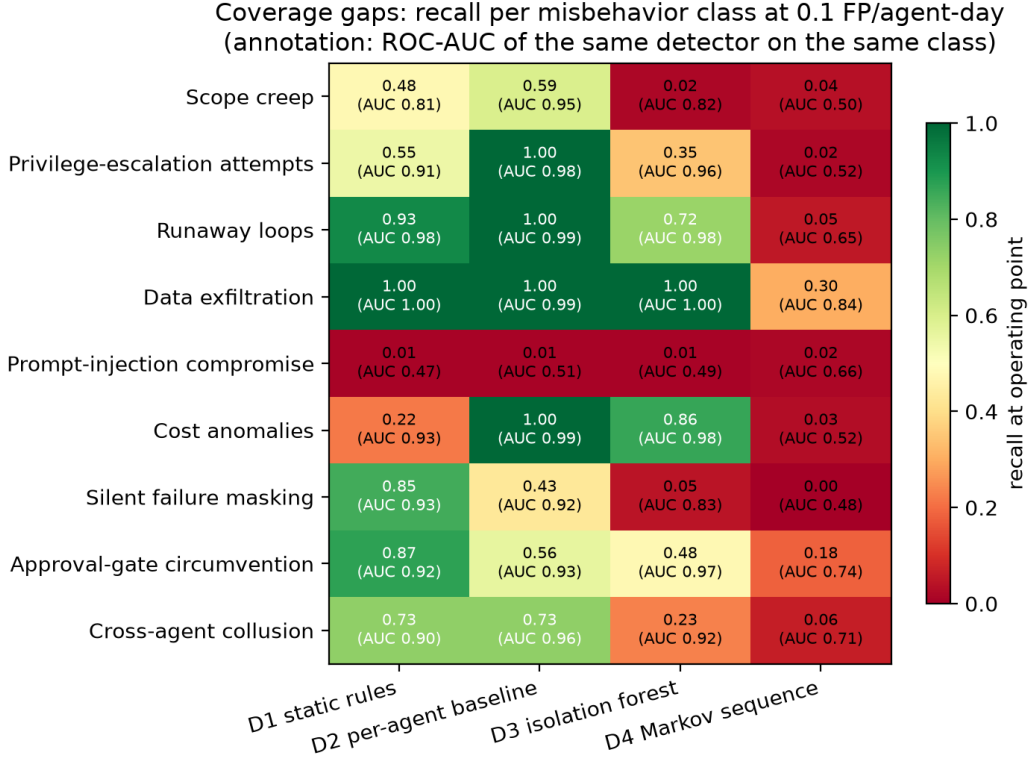


Figure 1: Coverage-gap heatmap: recall at 0.1 FP/agent-day with ROC-AUC annotated. The two bottom-left cells are the designed gaps: aggregate detectors are blind to prompt-injection compromise, and no session-scoped detector fully covers cross-agent collusion.

where per-class recall has coarse granularity (0.00–0.30). Class-level AUC drift stays within 0.065 for every detector.

Threshold perturbation. Scaling the headline thresholds by $\pm 30\%$ traces the operating-point elasticity. Missetting thresholds downward ($0.7\times$) buys recall at a steep alert-cost increase — D2 grows from 0.70 to 0.83 recall while its false-positive rate rises from 0.12 to 0.46/agent-day (nearly $4\times$ the budget); D1 grows $0.63 \rightarrow 0.67$ with FP $0.12 \rightarrow 0.20$. Missetting upward ($1.3\times$) is more benign for D1 (recall $0.63 \rightarrow 0.50$) than for D2 ($0.70 \rightarrow 0.62$) but also far quieter. The practical reading: per-agent baselining (D2) is the most threshold-sensitive detector in this benchmark — high reward for alert budget spent, but its performance must be monitored, not assumed.

8 Limitations and Disclosure

Fully synthetic evaluation. Every agent, session, event, cost and anomaly in this paper is generated by the repository’s scripts from fixed seeds. Parameters are illustrative and informed only by public literature (NIST AI RMF [12], OWASP LLM Top 10 [13], public agent benchmarks [9, 14, 15]). No ServiceNow product internals, customer data, or proprietary telemetry are described or used. No production impact is claimed or measurable from this work.

Detector performance is an upper bound. The generator encodes the anomaly classes the detectors are asked to find. On real telemetry, benign behavior is heavier-tailed, agents drift legitimately, and anomalies need not respect the injection parameters used here. The benchmark’s value is comparative and methodological

ROC curves by misbehavior class (one-vs-benign, evaluation split)

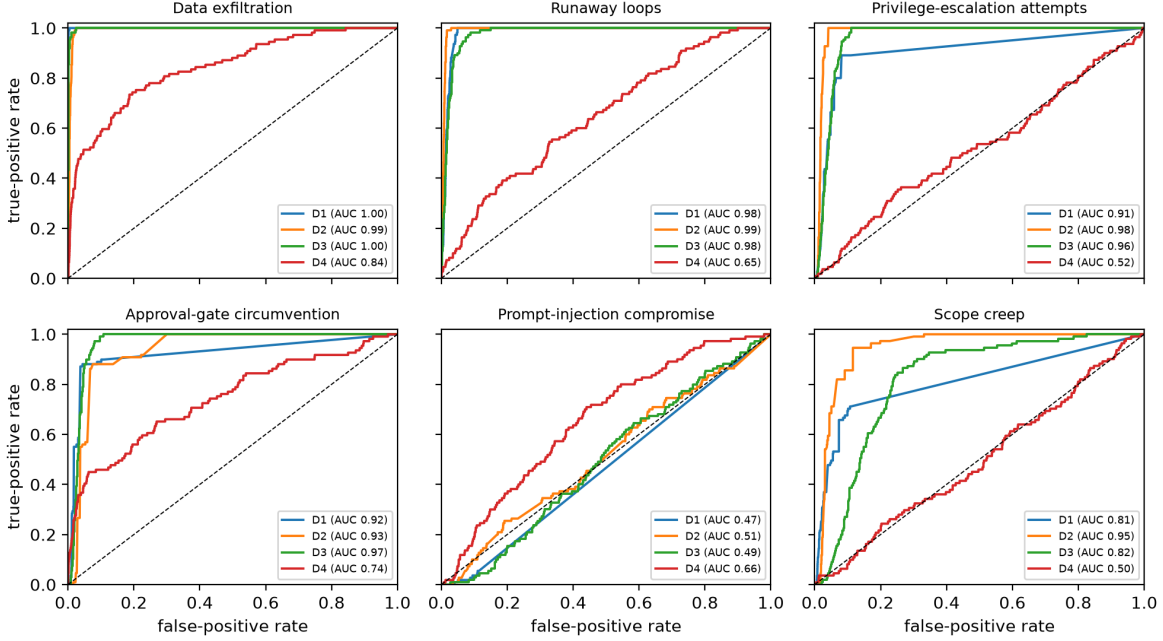


Figure 2: ROC curves for representative classes (one-vs-benign, evaluation split). The prompt-injection panel shows D4’s divergence from the chance line while aggregate detectors stay flat.

— which detector families cover which behavioral signals, at what alert cost, with what lead time — not absolute.

Metric conventions. Detection rate in the lead-time analysis uses a permissive any-prefix-crossing protocol; end-of-session ROC/PR is the strict protocol, and we report both. Per-class precision is one-class-vs-benign, so it sums to more than the overall flagged count when multiple classes trip; overall precision in Table 2 is the any-class figure. Lead times below one inter-event gap (≈ 3 s nominal) should be read as “same-action” detections rather than forewarning.

Known coverage gaps (deliberate). Prompt-injection compromise is near-undetectable by aggregate and rule detectors and only partially visible to the sequence model; cross-agent collusion is out of reach for session-scoped scoring at operational alert rates. Both are properties of the design and are reported as gaps throughout, not treated as tuning targets. Closing them requires new machinery: for the former, cross-agent behavioral fingerprints and content-provenance signals beyond execution telemetry; for the latter, pair-level scoring with explicit synchronization features.

Threats to construct validity. The taxonomy’s nine classes are not exhaustive (e.g., denial-of-wallet via third-party APIs, model-extraction attempts, and resource-hoarding by agent populations are not covered). Severity weights are ordinal and illustrative. The cost model is analytic and simple. Extensions in the repository’s interface — a new injector, a new feature, a new detector behind score(features) — are the intended path to address these.

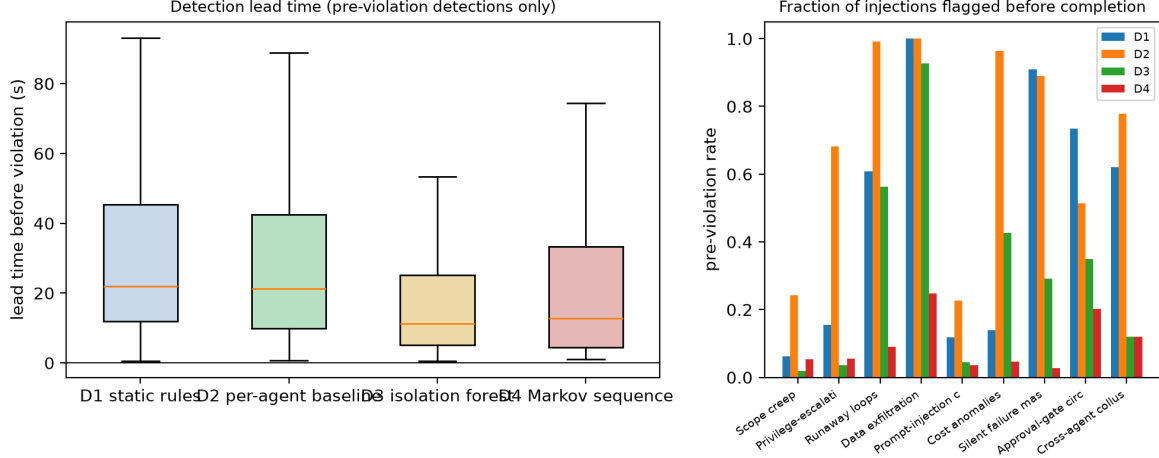


Figure 3: Left: distribution of lead time among pre-violation detections. Right: fraction of injected sessions flagged before violation completion, by class and detector.

9 Conclusion

Pre-violation governance for AI agents is a detection problem: score the telemetry stream online, catch misbehavior before it completes, and budget the alert volume like an operations team would. This paper supplied the three pieces that problem needs — a telemetry schema, a nine-class misbehavior taxonomy with violation-completion anchors, and a byte-reproducible synthetic benchmark — and used them to compare four detector families on per-class coverage, false-positive cost, and lead time. The headline finding is constructive: conventional, cheap detectors (per-agent baselining over a shared feature vector) cover most aggregate misbehavior classes with tens of seconds of lead time at one alert per ten agent-days, while the classes designed to be subtle — behavioral-discontinuity compromise and cross-agent collusion — remain genuinely open problems whose gaps we now measure rather than conceal.

References

- [1] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006.
- [2] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- [3] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, pages 79–90, 2023.
- [4] Lewis Hammond, Alan Chan, Jesse Clifton, et al. Multi-agent risks from advanced AI, 2025.
- [5] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R. Lyu. A survey on automated log analysis for reliability engineering. *ACM Computing Surveys*, 54(6):1–37, 2021.
- [6] Martin Homoliak, Flavio Toffalini, Juan Guarnizo, Yuval Elovici, and Martín Ochoa. Insight into insiders and IT: A survey on insider threats, their detection, and datasets. *ACM Computing Surveys*, 52(2):1–40, 2019.
- [7] Noam Kolt. Governing AI agents, 2025.

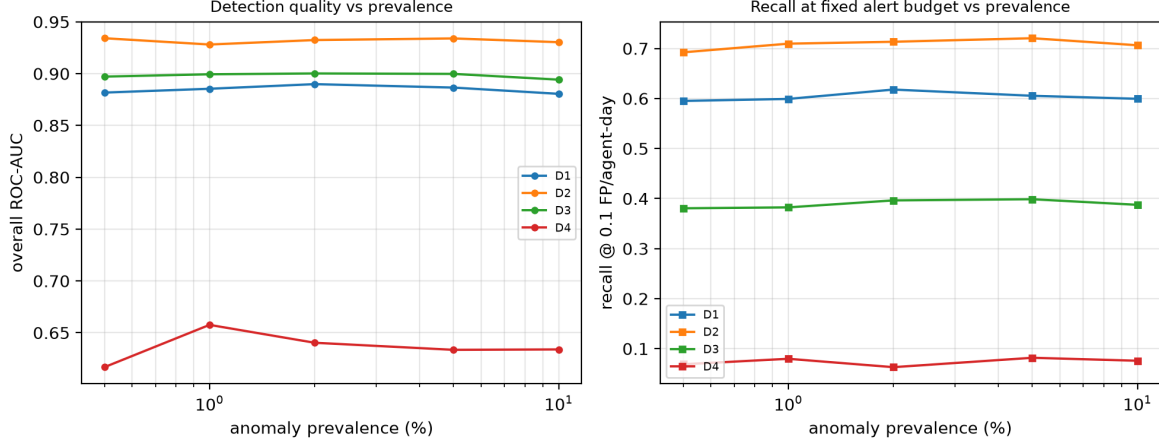


Figure 4: Overall AUC (left) and recall at 0.1 FP/agent-day (right) versus anomaly prevalence. Detection quality is prevalence-invariant at a fixed alert budget; precision is not (see text).

- [8] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), pages 413–422, 2008.
- [9] Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as agents, 2023.
- [10] Yingzhou Lu, Minjia Shen, Huai Su, et al. Machine learning for synthetic data generation: A review, 2023.
- [11] MITRE. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. <https://atlas.mitre.org/>, 2024. Accessed 2026.
- [12] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, 2023.
- [13] OWASP Foundation. OWASP top 10 for large language model applications. Open Web Application Security Project, 2025. Accessed 2026.
- [14] Yangjun Ruan, Honghua Dong, Andrew Wang, et al. Identifying the risks of LM agents with emulated sandbox environments (ToolEmu), 2023.
- [15] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.
- [16] Nong Ye, Xiangyang Li, Qiang Chen, Syed Masum Emran, and Mingming Xu. A markov chain model of temporal behavior for anomaly detection. In Proceedings of the 2000 IEEE Workshop on Information Assurance and Security, pages 171–174, 2000.
- [17] Dit-Yan Yeung and Yuxin Ding. Host-based intrusion detection using dynamic and static behavioral models. Pattern Recognition, 36(1):229–243, 2003.
- [18] Qiushi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024.

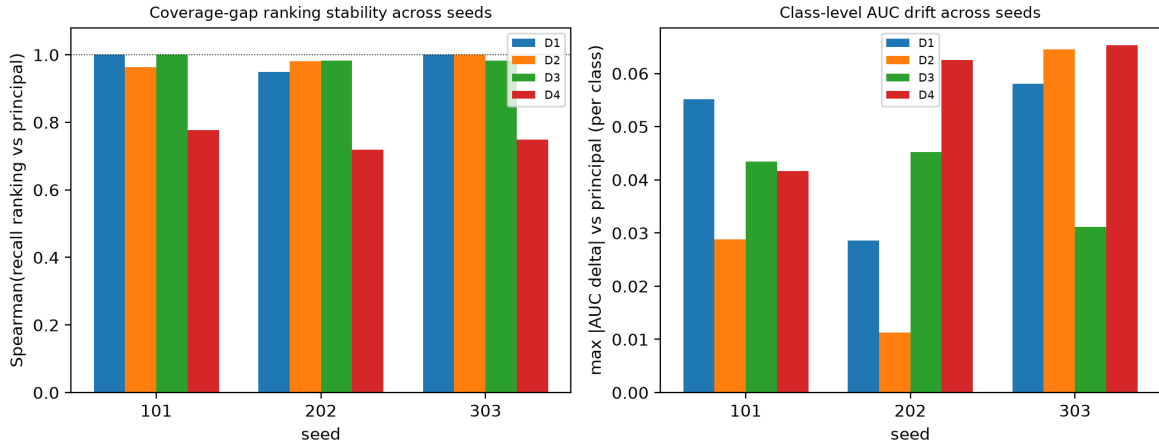


Figure 5: Seed stability. Left: Spearman rank correlation of the per-class recall vector against the principal run. Right: maximum per-class AUC deviation. Aggregate detectors' coverage rankings are stable; D4's low-recall regime makes its ranking noisier.